

Welcome and Introduction to Management Science

BUS 104: Decision Analysis and Management Science

Danko Turcic

September 17, 2026



Course Information

- **Instructor:** Danko Turcic, Associate Professor of Operations Management
- **Quarter:** Fall 2026
- **Office:** Anderson-1 421
- **Meeting Time:** MW 11:00 AM - 12:20 PM
- **Email:** danko.turcic@ucr.edu
- **Classroom:** SB 165
- **Website:** <https://eLearn.ucr.edu>
- **GitHub Repo:** <https://github.com/dxt48/BUS-104>
- **Office Hours:** M 12:30 - 1:30 PM (in person), Tue 11:00 AM - 12:00 PM (Zoom)

Vision

We pursue excellence in all that we do, as our vision is to be an internationally recognized leader in business education and research.

Mission

We create knowledge through impactful research, unlock potential through exceptional educational experiences, and foster success in an innovative and collaborative environment.

Strategy

We pursue excellence in all we do by cultivating our environment, engaging our community and empowering our stakeholders.

The course fosters analytical and quantitative approaches to solving business problems. Regardless of their functional orientation, many managerial decisions are increasingly based on analysis using quantitative models from management science. Management science tools, techniques and concepts (e.g., data, models, and software programs) have dramatically changed how businesses operate in manufacturing, services, marketing, finance, and other areas.

Course Objectives and Materials

Objectives

- Learn computer skills to solve decision analysis and management science problems
- Learn about decisions under certainty (deterministic math programming)
- Learn about modeling uncertainty making decisions under uncertainty (simulation)

Materials

- Lecture Notes: Electronic version posted on eLearn
- Helpful References:
 - B. Render, R.M. Stair Jr., and N. Balakrishnan, Managerial Decision Modeling with Spreadsheets
 - Hillier, F. S., and G. J. Lieberman: Introduction to Operations Research
 - Taha, H. A.: Operations Research: An Introduction

Grading Policy and Homework

Grading Components

Component	Weight
Mid-term exam	25%
Final exam	25%
Homework	40%
Online Case	10%

Grade Scale

Grades Scored Between	Letter Grade
97%	100% A+
94%	97% A
90%	94% A-
87%	90% B+
84%	87% B
80%	84% B-
77%	80% C+
74%	77% C
70%	74% C-
67%	70% D+
64%	67% D
60%	64% D-
0%	60% F

General Approach to the Course

- Focus on what you are getting out of the class
- It's okay if you don't get something right away
- Ask questions during class

Optimization Mind Set

- We have a set of decisions to make, how do we make them optimally?
 - Inventory: how many loaves of bread should a bakery bake each day?
 - Staffing: how many employees should a firm employ?
- There are issues that can complicate these decision problems.
 - Optimization is vague.
 - Parameter estimates come with uncertainty.
- We will learn mathematical tools to help us make optimal decisions.
 - Continuous/Integer Programming

- **Decision Variables:** Variables that represent our decisions
 - Example: How much to invest in Stock A
- **Objective Function:** The quantity that we are either trying to
 - Max: total return, total profit
 - Min: costs, wait time
- **Constraints:** Express restrictions on values that variables can take
 - Example: budget, space constraints

Exercise: Applying the Optimization Mindset

You want your daily diet plan to have at least 100 percent of the U.S. FDA of vitamin C, calcium; at least 50 grams of protein; and at most 2000 calories. You are wondering if this can be accomplished by eating McDonald's daily nutrition standard.

Menu Item	Price (\$)	McDonald's Food Facts						
		Cal	Protein	Fat	Vit. A	Vit. C	Calcium	Iron
Hamburger	0.69	250	13	90	2	2	15	2
Filet-O-Fish	1.83	400	14	60	2	2	15	2
Chicken Fries (6 pcs)	1.39	250	15	170	2	2	2	4
Garden Salad	2.05	35	2	0	120	40	4	6
Honey	0.79	250	13	20	0	40	2	6

* Prices recorded Jan. 12, 2000 in St. Louis, Missouri

Diet Problem is a Linear Program

Key Parts:

- **Decision Variables:** What you control (e.g., how many products to make).

Diet Problem is a Linear Program

Key Parts:

- **Decision Variables:** What you control (e.g., how many products to make).
- **Objective Function:** What to optimize (e.g., maximize profit = $20x + 30y$).

Diet Problem is a Linear Program

Key Parts:

- **Decision Variables:** What you control (e.g., how many products to make).
- **Objective Function:** What to optimize (e.g., maximize profit = $20x + 30y$).
- **Constraints:** Limits (e.g., resources like time or budget).

Diet Problem is a Linear Program

Key Parts:

- **Decision Variables:** What you control (e.g., how many products to make).
- **Objective Function:** What to optimize (e.g., maximize profit = $20x + 30y$).
- **Constraints:** Limits (e.g., resources like time or budget).
- Assumptions: Everything linear and deterministic (no randomness... yet!).
- Fun fact: Used by Google for ad placements and UPS for routes – saving millions!
- Think: How would you allocate your weekly budget for max fun?

LP Example: Factory Profit Max

Imagine your startup makes Gadgets (G) and Widgets (W):

- Profit: $20G + 30W$ (maximize!)
- Constraints:
 - Machine time: $2G + 3W \leq 12$ hours
 - Labor: $G + W \leq 5$ hours
 - $G, W \geq 0$ (no negatives!)
- Question: What's the optimal mix?
- Spoiler: $G=3, W=2$, Profit=120. (But how? Math magic!)
- Real-world vibe: Like baking cookies vs. cakes with limited oven time.

How is this problem similar to the Diet Problem we just considered?

LP Example: Factory Profit Max

Imagine your startup makes Gadgets (G) and Widgets (W):

- Profit: $20G + 30W$ (maximize!)
- Constraints:
 - Machine time: $2G + 3W \leq 12$ hours
 - Labor: $G + W \leq 5$ hours
 - $G, W \geq 0$ (no negatives!)
- Question: What's the optimal mix?
- Spoiler: $G=3, W=2, \text{Profit}=120$. (But how? Math magic!)
- Real-world vibe: Like baking cookies vs. cakes with limited oven time.

How is this problem similar to the Diet Problem we just considered?

LP in Python: Let's Code It!

Using `scipy.optimize.linprog` – super handy for quick solves.

```
from scipy.optimize import linprog

# Maximize 20G + 30W -> minimize -20G -30W
c = [-20, -30]

# Constraints: 2G+3W <=12, G+W<=5
A = [[2, 3], [1, 1]]
b = [12, 5]

# Bounds: >=0
bounds = [(0, None), (0, None)]

res = linprog(c, A_ub=A, b_ub=b, bounds=bounds)
print("Optimal G, W:", res.x) # [3. 2.]
print("Max Profit:", -res.fun) # 120.0
```

Challenge

- Try to implement the code yourself right now!!!!
- Tweak the numbers in your notebook – what happens if labor increases?

Integer Linear Programming: When Fractions Don't Cut It

- ILP: LP but variables must be whole numbers (integers).
- Why? Can't make 2.5 cars or hire 1.7 employees!
- Adds complexity – sometimes NP-hard, but powerful for scheduling, routing.
- Example: Assigning tasks to workers without splitting people.
- Engaging twist: Like packing your backpack for a hike – max value, integer items only!
- Research angle: At UCR, profs use ILP for supply chain optimizations in uncertain economies.

Integer Linear Programming: When Fractions Don't Cut It

- ILP: LP but variables must be whole numbers (integers).
- Why? Can't make 2.5 cars or hire 1.7 employees!
- Adds complexity – sometimes NP-hard, but powerful for scheduling, routing.
- Example: Assigning tasks to workers without splitting people.
- Engaging twist: Like packing your backpack for a hike – max value, integer items only!
- Research angle: At UCR, profs use ILP for supply chain optimizations in uncertain economies.

Integer Linear Programming: When Fractions Don't Cut It

- ILP: LP but variables must be whole numbers (integers).
- Why? Can't make 2.5 cars or hire 1.7 employees!
- Adds complexity – sometimes NP-hard, but powerful for scheduling, routing.
- Example: Assigning tasks to workers without splitting people.
- Engaging twist: Like packing your backpack for a hike – max value, integer items only!
- Research angle: At UCR, profs use ILP for supply chain optimizations in uncertain economies.

Integer Linear Programming: When Fractions Don't Cut It

- ILP: LP but variables must be whole numbers (integers).
- Why? Can't make 2.5 cars or hire 1.7 employees!
- Adds complexity – sometimes NP-hard, but powerful for scheduling, routing.
- Example: Assigning tasks to workers without splitting people.
- Engaging twist: Like packing your backpack for a hike – max value, integer items only!
- Research angle: At UCR, profs use ILP for supply chain optimizations in uncertain economies.

Integer Linear Programming: When Fractions Don't Cut It

- ILP: LP but variables must be whole numbers (integers).
- Why? Can't make 2.5 cars or hire 1.7 employees!
- Adds complexity – sometimes NP-hard, but powerful for scheduling, routing.
- Example: Assigning tasks to workers without splitting people.
- Engaging twist: Like packing your backpack for a hike – max value, integer items only!
- Research angle: At UCR, profs use ILP for supply chain optimizations in uncertain economies.

Integer Linear Programming: When Fractions Don't Cut It

- ILP: LP but variables must be whole numbers (integers).
- Why? Can't make 2.5 cars or hire 1.7 employees!
- Adds complexity – sometimes NP-hard, but powerful for scheduling, routing.
- Example: Assigning tasks to workers without splitting people.
- Engaging twist: Like packing your backpack for a hike – max value, integer items only!
- Research angle: At UCR, profs use ILP for supply chain optimizations in uncertain economies.

ILP Example: 0-1 Knapsack Adventure

You're hiking: Max value in backpack (weight ≤ 10 lbs). Items:

Item	Value	Weight
Tent	10	5
Stove	40	4
Water Filter	30	6

- Take (1) or leave (0) each item.
- Optimal: Stove + Water Filter (value 70, weight 10).
- Question: What if you add a sleeping bag (value 50, weight 7)? Recalculate!

ILP in Python: Binary Magic with PuLP

Using pulp – great for integer constraints.

```
import pulp

prob = pulp.LpProblem("Knapsack", pulp.LpMaximize)

# Binary vars: 1=take, 0=leave
x1 = pulp.LpVariable('Tent', cat='Binary')
x2 = pulp.LpVariable('Stove', cat='Binary')
x3 = pulp.LpVariable('Filter', cat='Binary')

# Objective
prob += 10*x1 + 40*x2 + 30*x3

# Constraint
prob += 5*x1 + 4*x2 + 6*x3 <= 10

prob.solve()
print(pulp.value(x1), pulp.value(x2), pulp.value(x3)) # 0 1 1
```

- Pro tip: Run this – experiment with more items for your "hike"!

Simulation: Embracing the Random World

- When life throws randomness (demand, weather, stocks) – LP/ILP assume certainty.
- Simulation: Model systems with random inputs, run many trials, analyze stats.
- Monte Carlo method: Use randomness to solve problems (named after casino!).
- Applications: Risk analysis in finance, queue wait times at Disneyland.
- Why engaging? Predict the unpredictable – like simulating your fantasy football league!
- UCR research: Simulating climate impacts on supply chains.

Simulation Example: Estimating Pi with Darts

Throw virtual darts at a square (-1 to 1 in x,y).

- If dart lands inside unit circle ($x^2 + y^2 \leq 1$), count it.
- $\pi \approx 4 * (\text{hits} / \text{total throws})$
- Why? Area of circle / area of square = $\pi/4$.
- Fun: More throws = better estimate. Try 10 vs. 10,000!
- Real-world: Similar to predicting election outcomes with polls.

Simulation in Python: Monte Carlo Pi

Using numpy for random numbers – easy and fast.

```
import numpy as np

def estimate_pi(n):
    inside = 0
    for _ in range(n):
        x, y = np.random.uniform(-1, 1, 2)
        if x**2 + y**2 <= 1:
            inside += 1
    return 4 * inside / n

print(estimate_pi(10000))  # ~3.14 (varies slightly!)
```

- Challenge: Plot the points for visualization (add matplotlib!).
- Wrap-up: These tools empower you – code, optimize, simulate your way to success! Questions?

Structure of the Class

- **Bring a computer to each class!!!!** Chromebook is okay; iPad will most likely not work.
- Presentation to teach you new skills and techniques - we will keep building and building... How should I approach these lectures?
- Weekly homework will test your progress:
 - **Homework:** Posted on Canvas, late submissions incur 10% penalty per day, no submissions after one week.
 - **Two attempts:** eLearn will grade the **last** attempt
 - There will be no homework the first week...